

Reverse Engineering and Runtime Manipulation of Legacy Game Software

Author: Dilsara Hansaja

Date: 04 April 2026

Table of Contents

1	Game Overview and Environment Setup	3
1.1	Game Justification	3
1.2	Platform Sandbox.....	3
1.3	Tools Setup	4
2	Methodology	4
3	Reverse Engineering and Analysis	5
3.1	Static Analysis	5
3.2	Dynamic Analysis	6
3.3	Precise Register and Memory Identification.....	7
4	Assembly level Modification	8
4.1	Instruction Identification.....	8
4.2	Modification Implementation	8
4.3	Technical Explanation of the Changes	9
5	Security Analysis and Discussion	9
5.1	Vulnerability Discussion.....	9
5.2	Proposed Mitigation Strategies	10
6	Conclusion	10
7	References.....	10

1 Game Overview and Environment Setup

1.1 Game Justification

For this analysis, the legacy software selected is **Project I.G.I. (I'm Going In)**, a first-person shooter game developed by Innerloop Studios and released in 2000 by Edios Interactive. Furthermore, it is strictly an offline, single-player application with no online services or anti-cheat integrations.

From a cybersecurity perspective, Project I.G.I. serves as an ideal candidate for reverse engineering. The game was developed using C and C++, which can be easily analyzed via debugging tools. As legacy software, the game lacks foundational security mechanisms such as Address Space Layout Randomization (ASLR), code obfuscation, and active memory protection. These vulnerabilities allow us to conduct predictable memory mapping and unrestricted runtime manipulation, making it highly suitable for assembly-level analysis.

1.2 Platform Sandbox

To ensure a secure and stable environment for memory analysis, a **Windows 10 Virtual Machine (VM) hosted on VMware** was utilized as the primary platform sandbox. The testing environment achieved two critical objectives:

1. **Security Isolation:** It physically separated the active manipulation of executable memory from the host operating system.
2. **Tool Compatibility:** It provided a highly stable environment necessary to run debugging and decompilation tools without causing system crashes.

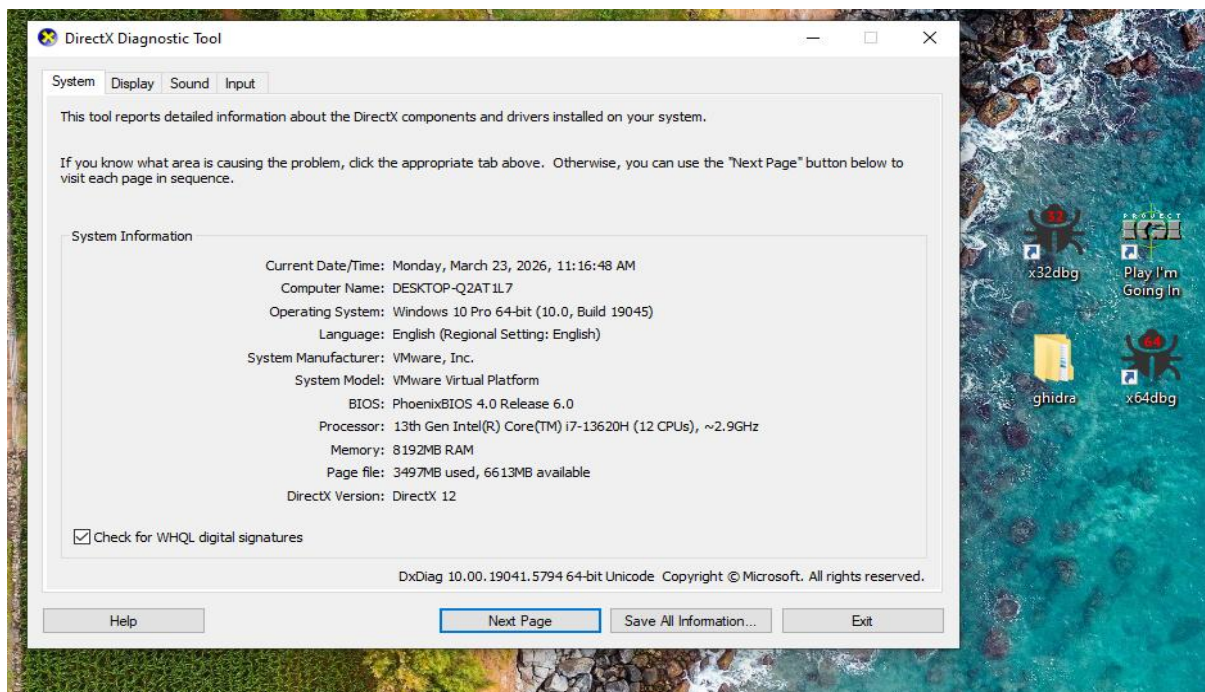


Figure 1: Sandbox setup using VMware

1.3 Tools Setup

The reverse engineering was conducted using two distinct analysis tools, each selected for a specific phase of the exploit chain:

- **Ghidra:** Selected as the primary static analysis debugger. Ghidra was used to decompile the legacy executable, convert assembly code into readable C-like logic, and identify the function structures controlling weapon-firing mechanics.
- **x32dbg:** Selected as the dynamic debugger to make changes during runtime verification after the static code was identified using Ghidra. The x32dbg was attached to the game process to track execution flows, create breakpoints, and identify the exact assembly instructions responsible for modifying the variable. It was further used to perform the runtime patch.

2 Methodology

The objective of this analysis was to identify and manipulate the runtime memory handling of the **player's ammo variable**. This was achieved by changing the relevant assembly instruction from **DEC** to **NOP**, which would allow players to shoot an infinite number of bullets throughout the game.

To achieve this without triggering system instability, a four-phase reverse engineering methodology was performed:

1. **Static Analysis:** The executable was first analyzed using Ghidra to examine the disassembled code and reverse-engineer the ammo decrement logic back to C code. Then, the specific instructions responsible for controlling ammunition reduction were identified.
2. **Dynamic Analysis:** Once the memory address was identified using Ghidra, hardware breakpoints were applied using x32dbg to monitor access to the address. This allowed the identification of the exact assembly instruction responsible for modifying the ammo value.
3. **Memory and Register Verification:** After identifying the instruction, the memory pointer and registers handling the ammunition values were analyzed in real-time using x32dbg's memory dump to confirm their function during live gameplay.
4. **Applying the Patch:** Finally, the identified instruction was modified at runtime by changing the DEC instruction with NOP using x32dbg. This prevented the decrement operation, effectively enabling infinite ammunition during gameplay.

3 Reverse Engineering and Analysis

3.1 Static Analysis

The analysis was started by conducting the static reverse engineering process to locate the weapon logic. This was done by disassembling the game executable IGI.exe within Ghidra and then developing the corresponding high-level C code logic from the assembly. By identifying the interface within the game, the instructions used to handle the ammunition count were isolated.

The static analysis of these instructions revealed the execution flow of the ammunition variable using the memory pointer **[EBP+0x144]**, The program then checks the ammo value is greater than zero using an “if condition” and performs the subtraction of the ammunition value using the instruction **DEC EAX**. The instruction responsible for the decrement instruction can be identified by the address **00479A9E**.

```
00479a8a 8b 85 2c    MOV     EAX,dword ptr [EBP + 0x12c]
01 00 00
00479a90 85 c0       TEST    EAX,EAX
00479a92 74 11       JZ      LAB_00479aa5
00479a94 8b 85 44    MOV     EAX,dword ptr [EBP + 0x144]
01 00 00
00479a9a 85 c0       TEST    EAX,EAX
00479a9c 7e 07       JLE     LAB_00479aa5
00479a9e 48          DEC     EAX
00479a9f 89 85 44    MOV     dword ptr [EBP + 0x144],EAX
01 00 00

LAB_00479aa5                                XREF[2]: 00479a92(j), 00479a9c(j)
00479aa5 e8 66 d8    CALL    FUN_004e7310
06 00
00479aaa 89 85 38    MOV     dword ptr [EBP + 0x138],EAX
01 00 00
```

Figure 2: Identified function structure controlling weapon-firing mechanics using Ghidra

The assembly breakdown of the analyzed instructions is as follows:

1. **00479A94** **MOV EAX, dword ptr[ebp+0x144]**- The game loads the current ammo value into the EAX register.
2. **00479A9A** **TEST EAX, EAX**- This checks whether the value in EAX is zero.
3. **00479A9C** **JLE LAB_00479aa5**- If the ammo value is zero or less, execution jumps to **LAB_00479aa5**, skipping the decrement operation.
4. **00479A9E** **DEC EAX**- This instruction decreases the ammo value by one.
5. **00479A9F** **MOV dword ptr[EBP+0x144], EAX**- The updated ammo value in EAX is written back to memory.

Static analysis of the decompiled function revealed a loop that the game uses to store and update the ammunition value. By analyzing the assembly code, the following high-level **C program logic** was developed (Figure 3).

```

1  // [ebp+0x144] store players current ammo count
2  int currentAmmoCount = playerState.currentAmmoCount;
3
4  // test eax, eax
5  //jle: prevents against zero or negative ammo
6  // If ammo is zero or negative, skip the decrement
7  if (currentAmmoCount > 0)
8  {
9      // dec eax
10     // Decrease the ammo count by one after firing
11     currentAmmoCount--;
12
13     // mov [ebp+0x144], eax
14     // Store the updated ammo value back into memory
15     playerState.currentAmmoCount = currentAmmoCount;
16 }
17
18 // Continue with weapon firing logic
19 fireWeapon();

```

Figure 3: Reverse Engineered C program logic

3.2 Dynamic Analysis

After understanding program logic, the next step was to verify static findings dynamically using a debugger. To perform this, the game was attached to x32dbg.

Systematic Execution Tracing

To capture the game exactly when it changes the ammunition value, a **breakpoint** was set on the identified instruction 00479A9E *dec eax* (Figure 4). The use of breakpoint ensured that execution paused exactly when it changes ammunition value was modified, allowing precise identification of the responsible instruction *igi.exe*.

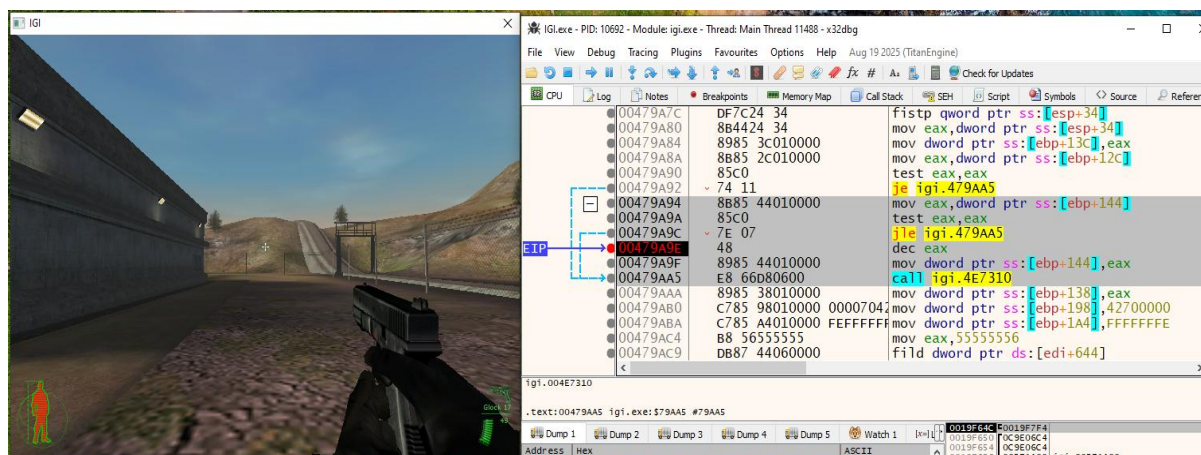
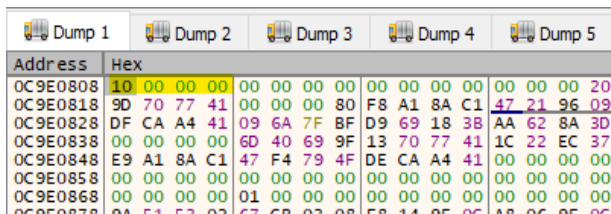


Figure 4: x32dbg CPU window showing the breakpoint hit when player shoots.

Dynamic Memory Tracking

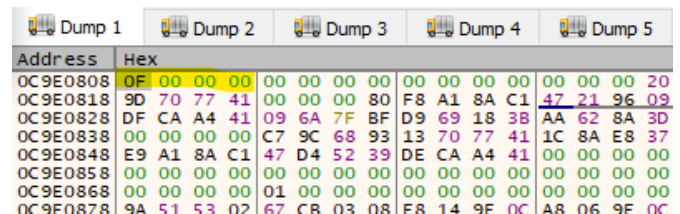
To prove that the selected memory controls the ammunition changes dynamically, the pointer **[ebp+144]** was isolated and brought to the memory dump window of the x32dbg application.

By actively firing the weapon in the game, the hexadecimal value located at the obtained memory dump was tracked. It was observed that the value decreased continuously with each shot fired within the game. (e.g. **The hexadecimal located in memory dump 10 00 00 00 which resulted to 16 in decimal was reduced to 0F 00 00 00 which resulted to 15 in decimal**) (Fig. 5, Fig. 6). By analyzing the memory dump, it was confirmed that the dump represents the current ammunition count of the weapon the player holds.



Dump 1	Dump 2	Dump 3	Dump 4	Dump 5
Address	Hex			
0C9E0808	10 00 00 00	00 00 00 00	00 00 00 00	00 00 00 20
0C9E0818	9D 70 77 41	00 00 00 80	F8 A1 8A C1	47 21 96 09
0C9E0828	DF CA A4 41	09 6A 7F BF	D9 69 18 38	AA 62 8A 3D
0C9E0838	00 00 00 00	6D 40 69 9F	13 70 77 41	1C 22 EC 37
0C9E0848	E9 A1 8A C1	47 F4 79 4F	DE CA A4 41	00 00 00 00
0C9E0858	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0C9E0868	00 00 00 00	01 00 00 00	00 00 00 00	00 00 00 00

Fig. 5: x32dbg memory dump window highlighting Hex value 10 (Decimal 16) of the ammo



Dump 1	Dump 2	Dump 3	Dump 4	Dump 5
Address	Hex			
0C9E0808	0F 00 00 00	00 00 00 00	00 00 00 00	00 00 00 20
0C9E0818	9D 70 77 41	00 00 00 80	F8 A1 8A C1	47 21 96 09
0C9E0828	DF CA A4 41	09 6A 7F BF	D9 69 18 38	AA 62 8A 3D
0C9E0838	00 00 00 00	C7 9C 68 93	13 70 77 41	1C 8A E8 37
0C9E0848	E9 A1 8A C1	47 D4 52 39	DE CA A4 41	00 00 00 00
0C9E0858	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0C9E0868	00 00 00 00	01 00 00 00	00 00 00 00	00 00 00 00
0C9F0878	9A 51 53 02	67 C8 03 08	F8 14 9F 0C	A8 06 9F 0C

Fig. 6: x32dbg memory dump window highlighting Hex value 0F (Decimal 15) of the ammo

3.3 Precise Register and Memory Identification

By pausing the execution of the game, the relationship between the CPU registers and system RAM was identified.

- **Register (eax):** In x86 assembly architecture, the eax register is used as the primary register to perform mathematical operations. By inspecting the register on x32dbg, it was confirmed that the game temporarily loads the current ammunition count into eax allowing the CPU to perform the *dec eax* (subtract 1) operation without crashing the application.
- **Memory Location ([ebp+144]):** While the eax register handles the mathematical operations, the persistent ammunition data is stored within the system RAM. The assembly instruction executing after the decrement instruction (*mov dword ptr ss:[ebp+144], eax*) demonstrates that the game stores the new calculated eax value into a memory offset at 144 bytes away from the current **Base Pointer (ebp)**.

4 Assembly level Modification

4.1 Instruction Identification

Upon analyzing the instructions, the exact operation responsible for deducting ammunition was identified, which was located in the memory address **00479A9E** (Figure 7).

The instruction at this address was **dec eax (Decrement)**. When the weapon is fired, the game loads the current ammunition count from RAM into the eax register, and by using the dec instruction, the instruction subtracts one from the value stored in the EAX register exactly. As a result, the program writes the new value to the memory address.



Figure 7: Identifying dec instruction responsible for ammo reduction

4.2 Modification Implementation

To achieve the objective of changing the program behavior to hold the ammunition value constant, runtime manipulation was applied using x32dbg. This was performed by replacing the **selected dec eax** instruction with a **NOP** instruction. The modification implemented to the program is demonstrated by the image below (Figure 8).

00479A9A	85C0	7E 07	mov eax,word ptr ss:[ebp+144]
00479A9C	7E 07		test eax,eax
00479A9E	90		jle igi.479AA5
00479A9F	8985 44010000		nop
00479AA5	E8 66D80600		mov dword ptr ss:[ebp+144],eax
			call igi.4E7310

Figure 8: Applying the patch with NOP



Figure 9: Ammo Reduced before Patch Applied



Figure 10: Ammo Constant after patch Applied

Before applying the patch, firing the weapon resulted in a decrease in ammunition count (Figure 9). However, after replacing the dec eax instruction with nop, the ammunition value remained constant even after multiple shots (Figure 10). This confirms that the decrement operation was successfully bypassed.

4.3 Technical Explanation of the Changes

In x86 assembly, a NOP (No Operation) instructs the CPU to perform no action and simply execute the next instruction available.

By replacing the subtraction logic with NOP, the game bypasses the decrement operation without affecting overall program execution. The applied modification allows the game to still load the current ammunition into the `eax` register and verify it is greater than zero. However, since the instruction responsible for subtraction is replaced, the unchanged `eax` value is written directly back to the `[ebp+144]` memory offset.

This successfully forces the game to constantly overwrite the ammunition memory address with its own value, resulting in infinite ammunition.

5 Security Analysis and Discussion

5.1 Vulnerability Discussion

The runtime manipulation demonstrated in this report was successful as legacy games, including Project I.G.I operated within the user's local device without implementing any integrity controls that prevent users from modifying executable instructions.

By comparing the executable with **OWASP Architectural Principles for reverse engineering prevention**, three critical failures within the application architecture were revealed:

- 1. Absence of Anti-Debug Controls:** The application demonstrates the lack of environmental awareness as the x32dbg was attached to the IGI.exe, The game fails to detect the intrusion. As a secure application, it must implement Anti-Debug controls to detect external processes and terminate them to prevent information gathering.
- 2. Lack of Obfuscation and String Controls:** The application does not implement any form of code obfuscation or string protection. As a result, important strings related to weapon functionality could be easily identified using Ghidra's cross-referencing features. This made it possible to quickly locate the core logic responsible for ammunition handling. Implementing obfuscation techniques would make this analysis more difficult and slow down reverse engineering.
- 3. Missing Checksum Controls:** The game does not verify its own memory during runtime. As the instructions such as `dec eax` were replaced with `nop`, the application continued the execution without any change, suggesting that the program lacks checksum verification of the executed instructions to verify whether instructions have been modified after loaded into the RAM.

5.2 Proposed Mitigation Strategies

To prevent this type of static analysis, dynamic tracing, and assembly modification, modern systems should implement following security controls. In addition, a system should follow a defense in depth approach to ensure protection from multiple layers.

- **Address Space Layout Randomization (ASLR):** ASLR is a fundamental security feature that randomly arranges positions of key data areas such as the stack and heap within the operating system. Implementing ASLR ensures that static memory addresses change each time the program is launched, resulting the attacker to recalculate the execution flow and make sure hardcoded assembly patches are unstable.
- **Kernel-Level Anti-Debugging Defenses:** Secure applications use kernel-level drivers to prevent debugging. These drivers prevent access to tools that are being used by users, making it difficult for debugging tools like x32dbg to read and modify memory or to set breakpoints. Implementation of kernel-level anti-debugging further helps to prevent dynamic analysis of the application.
- **Implementation of Checksum & Dynamic Repair Controls:** To mitigate source code modifications, it is crucial for an application to use continuous integrity verification. By implementing controls like checksum, the application achieves the ability to frequently hash its active memory and compare it with a cryptographic signature. Therefore, if an attacker modifies the program with malicious intent, the program would be able to repair modified bytes using Dynamic repair control either by overwriting the malicious code or terminating the entire application.

6 Conclusion

In conclusion, this report demonstrated how reverse engineering techniques can be used to analyze and modify the ammunition logic in Project I.G.I. Using Ghidra and x32dbg, the instructions responsible for decreasing the ammo value were identified and modified to maintain a constant value. This shows how the lack of security controls in legacy applications allows runtime manipulation and highlights the need for stronger protection mechanisms.

7 References

- [1] OWASP Foundation. *Architectural Principles That Prevent Code Modification or Reverse Engineering* [Online]. Available: https://wiki.owasp.org/index.php/Architectural_Principles_That_Prevent_Code_Modification_or_Reverse_Engineering#Controls
- [2] National Security Agency. *Ghidra Software Reverse Engineering Framework*. Retrieved from <https://github.com/NationalSecurityAgency/ghidra>
- [3] x64dbg. *x64dbg Official Documentation*. Retrieved from <https://help.x64dbg.com/en/latest/>